

Question No:10

Question description

Kaaleppi has just robbed a bank and is now heading to the harbor. However, the police wants to stop him by closing some streets of the city.

What is the minimum number of streets that should be closed so that there is no route between the bank and the harbor?

Constraints

- $2 \leq n \leq 500$
- $1 \leq m \leq 1000$
- $1 \leq a, b \leq n$

Input

The first input line has two integers n and m : the number of crossings and streets. The crossings are numbered $1, 2, \dots, n$. The bank is located at crossing 1 , and the harbor is located at crossing n .

After this, there are m lines that describe the streets. Each line has two integers a and b : there is a street between crossings a and b . All streets are two-way streets, and there is at most one street between two crossings.

Output

First print an integer k : the minimum number of streets that should be closed. After this, print k lines describing the streets. You can print any valid solution.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 5
1 2
1 3
2 3
3 4
1 4
```

EXPECTED OUTPUT

```
2
1 4
3 4
```

Test Case 2

INPUT (STDIN)

```
4 5
1 1
2 2
2 3
3 4
1 4
```

EXPECTED OUTPUT

```
1
1 4
```

Code:

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAXN 505
5  #define MAXE 5005
6  #define INF 1000000000
7
8  typedef struct
9  {
10     int to;
11     int capacity;
12     int rev;
13 } Edge;
14
15 Edge graph[MAXN][MAXE];
16 int degree[MAXN];
17
18 int level[MAXN];
19 int ptr[MAXN];
20
21 int n, m;
22
23 /* Store the original streets */
24 int streetU[MAXE];
25 int streetV[MAXE];
26
27 /* Add a directed edge */
28 void addDirectedEdge(int u, int v, int capacity)
29 {
```

```

30     graph[u][degree[u]].to = v;
31     graph[u][degree[u]].capacity = capacity;
32     graph[u][degree[u]].rev = degree[v];
33
34     graph[v][degree[v]].to = u;
35     graph[v][degree[v]].capacity = 0;
36     graph[v][degree[v]].rev = degree[u];
37
38     degree[u]++;
39     degree[v]++;
40 }
41
42 /*
43  * An undirected street with capacity 1 is represented
44  * by two directed edges, each with capacity 1.
45  */
46 void addStreet(int u, int v)
47 {
48     addDirectedEdge(u, v, 1);
49     addDirectedEdge(v, u, 1);
50 }
51
52 /* BFS for Dinic */
53 int bfs(int source, int sink)
54 {
55     int queue[MAXN];

```

```

56     int front = 0;
57     int rear = 0;
58
59     for (int i = 1; i <= n; i++)
60         level[i] = -1;
61
62     level[source] = 0;
63     queue[rear++] = source;
64
65     while (front < rear)
66     {
67         int u = queue[front++];
68
69         for (int i = 0; i < degree[u]; i++)
70         {
71             Edge *e = &graph[u][i];
72
73             if (e->capacity > 0 && level[e->to] == -1)
74             {
75                 level[e->to] = level[u] + 1;
76                 queue[rear++] = e->to;
77             }
78         }
79     }
80
81     return level[sink] != -1;

```

```

82 }
83
84 /* DFS for Dinic */
85 int dfs(int u, int sink, int flow)
86 {
87     if (u == sink)
88         return flow;
89
90     for (int *i = &ptr[u]; *i < degree[u]; (*i)++)
91     {
92         Edge *e = &graph[u][*i];
93
94         if (e->capacity > 0 &&
95             level[e->to] == level[u] + 1)
96         {
97             int possibleFlow =
98                 (flow < e->capacity) ? flow : e->capacity;
99
100             int pushed = dfs(e->to, sink, possibleFlow);
101
102             if (pushed > 0)
103             {
104                 e->capacity -= pushed;
105                 graph[e->to][e->rev].capacity += pushed;
106
107                 return pushed;
108             }
109         }
110     }
111     return 0;
112 }

```

```

108         }
109     }
110 }
111
112     return 0;
113 }
114
115 /* Maximum flow */
116 int maxFlow(int source, int sink)
117 {
118     int flow = 0;
119
120     while (bfs(source, sink))
121     {
122         for (int i = 1; i <= n; i++)
123             ptr[i] = 0;
124
125         while (1)
126         {
127             int pushed = dfs(source, sink, INF);
128
129             if (pushed == 0)
130                 break;
131
132             flow += pushed;
133         }
134     }
135
136     return flow;
137 }
138
139 /*
140  After max-flow, find all vertices reachable from
141  source in the residual graph.
142  */
143 void findReachable(int source, int visited[])
144 {
145     int queue[MAXN];
146     int front = 0;
147     int rear = 0;
148
149     visited[source] = 1;
150     queue[rear++] = source;
151
152     while (front < rear)
153     {
154         int u = queue[front++];

```

```

154         int u = queue[front++];
155
156         for (int i = 0; i < degree[u]; i++)
157         {
158             Edge *e = &graph[u][i];
159
160             if (e->capacity > 0 && !visited[e->to])
161             {
162                 visited[e->to] = 1;
163                 queue[rear++] = e->to;
164             }
165         }
166     }
167 }
168
169 int main()
170 {
171     scanf("%d %d", &n, &m);
172
173     for (int i = 1; i <= n; i++)
174         degree[i] = 0;
175
176     /* Read streets */
177     for (int i = 0; i < m; i++)
178     {
179         int a, b;

```

```

180
181     scanf("%d %d", &a, &b);
182
183     streetU[i] = a;
184     streetV[i] = b;
185
186     addStreet(a, b);
187 }
188
189 /*
190 |     Maximum flow = minimum cut.
191 */
192 int answer = maxFlow(1, n);
193
194 /*
195 |     Find the source side of the minimum cut.
196 */
197 int visited[MAXN] = {0};
198
199 findReachable(1, visited);
200
201 /*
202 |     A street belongs to the minimum cut if one
203 |     endpoint is reachable from 1 and the other isn't.
204 */
205 int cutEdges[MAXE];
206 int cutCount = 0;
207

```

```

208     for (int i = 0; i < m; i++)
209     {
210         int u = streetU[i];
211         int v = streetV[i];
212
213         if (visited[u] != visited[v])
214         {
215             cutEdges[cutCount++] = i;
216         }
217     }
218
219     printf("%d\n", answer);
220
221     for (int i = 0; i < cutCount; i++)
222     {
223         int id = cutEdges[i];
224
225         printf("%d %d\n", streetU[id], streetV[id]);
226     }
227
228     return 0;
229 }

```

Output:

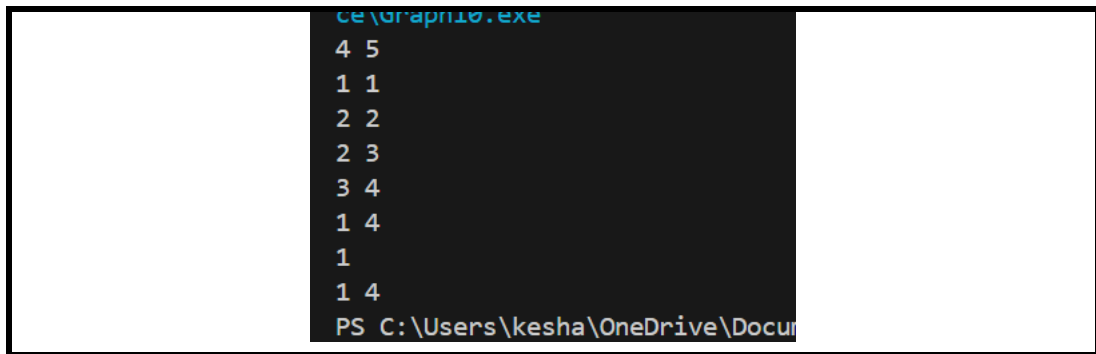
Test Case-1:

```

4 5
1 2
1 3
2 3
3 4
1 4
2
3 4
1 4
PS C:\Users\kesha\OneDrive\Documents

```

Test Case-2:



```
ce\graph10.exe
4 5
1 1
2 2
2 3
3 4
1 4
1
1 4
PS C:\Users\kesha\OneDrive\Docu
```

Keshav Kumar Rauniyar

CH.SC.U4CSE25236